logique pour ia

Séance 1 — Introduction au raisonnement logique pour l'IA

Chapitre 1 · Niveau débutant

Objectif : comprendre comment une IA utilise des faits et des règles pour produire des conclusions logiques, puis appliquer cela avec Python et SymPy.

1. Énoncé complet — Exercice 01 du TD

1. Un serveur est-il en panne ?

Tous les serveurs en panne sont indisponibles. S1 est en panne. S2 est indisponible.

Questions :

1. Que peux-tu affirmer sur S1 ?
2. Peux-tu conclure que S2 est en panne ? Propose une autre explication.
3. Quelles conclusions sont certaines et lesquelles restent seulement possibles ?

2. Cours — Faits, règles et déduction

En intelligence artificielle symbolique, le raisonnement logique consiste à utiliser des connaissances pour déduire de nouvelles informations.

On distingue trois éléments :

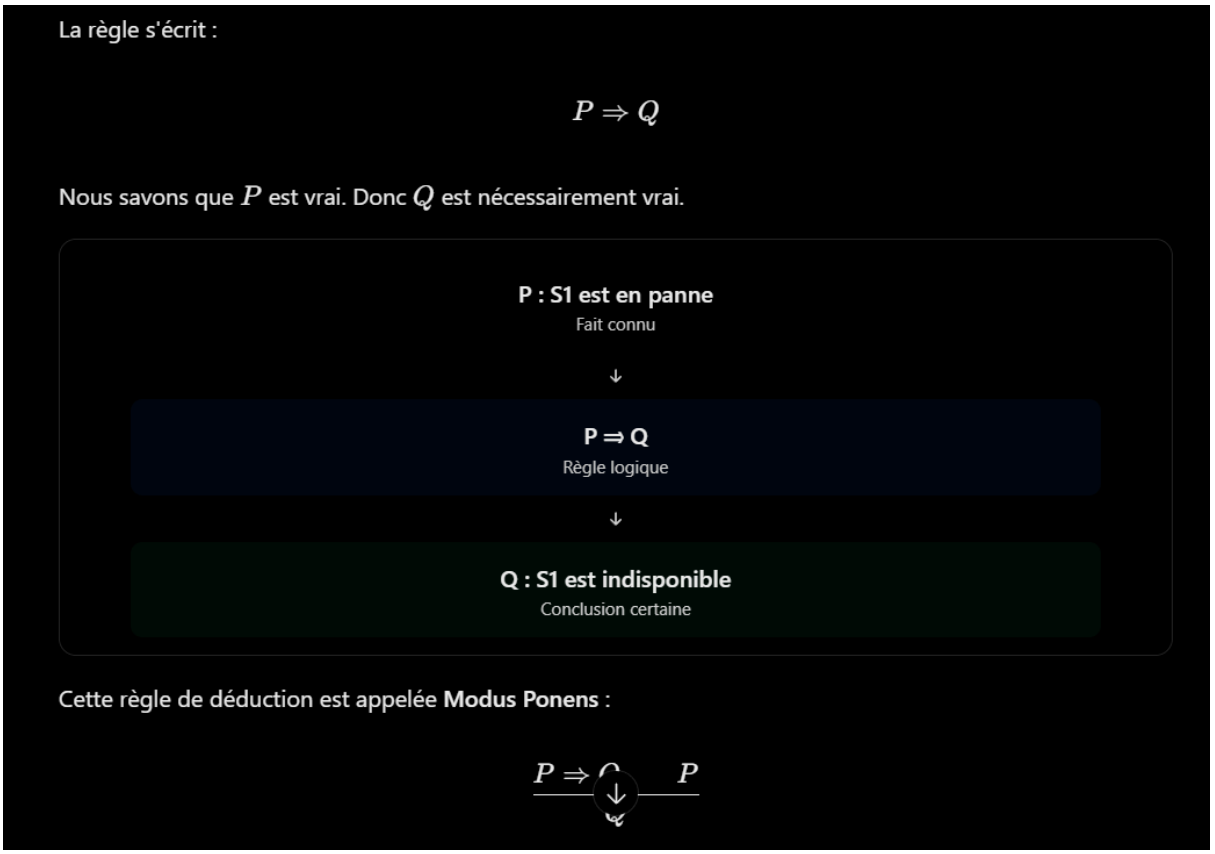
Élément	Définition	Exemple
Fait	Information connue	S1 est en panne
Règle	Relation logique	Panne ⇒ Indisponible
Conclusion	Information déduite	S1 est indisponible

Le raisonnement déductif

Nous représentons :

- (P) : S1 est en panne.

- (Q) : S2 est indisponible.



Une erreur logique fréquente
Pour S_2, nous savons uniquement qu'il est indisponible.
Peut-on affirmer qu'il est en panne ? Non.
En effet, la règle $P \Rightarrow Q$ ne signifie pas que $Q \Rightarrow P$.
S_2 pourrait être indisponible à cause d'une maintenance ou d'une coupure réseau.
On distingue donc :

- Conclusion certaine : elle découle nécessairement des faits et des règles.
- Conclusion possible : elle est compatible avec les faits, mais n'est pas démontrée.

1. La table de vérité de l'implication
   Avant de programmer, il faut comprendre comment fonctionne $P \Rightarrow Q$.

P	Q	$P \Rightarrow Q$
Vrai	Vrai	Vrai
Vrai	Faux	Faux

P	Q	$P \Rightarrow Q$
Faux	Vrai	Vrai
Faux	Faux	Vrai

Une implication est fausse uniquement lorsque P est vrai et Q est faux.
Par exemple, si le serveur est en panne mais reste disponible, cela contredit notre règle.

4. Pratique Python — SymPy

voir notebook logiquesympy

1. Énoncé complet — Exercice 02 du TD

02. Un badge et une porte

Si un badge est valide, la porte s'ouvre. Lina présente un badge valide. La porte est aussi ouverte quand un agent de sécurité l'ouvre manuellement.

Questions :

1. La porte s'ouvre-t-elle pour Lina selon les informations données ?
2. Si la porte est ouverte, peut-on affirmer qu'un badge valide a été présenté ?
3. Propose un contre-exemple précis.

2. Comprendre la logique avant de programmer

On commence par transformer les phrases en propositions logiques.

Symbole	Signification
B	Le badge est valide
P	La porte est ouverte
A	Un agent ouvre manuellement la porte

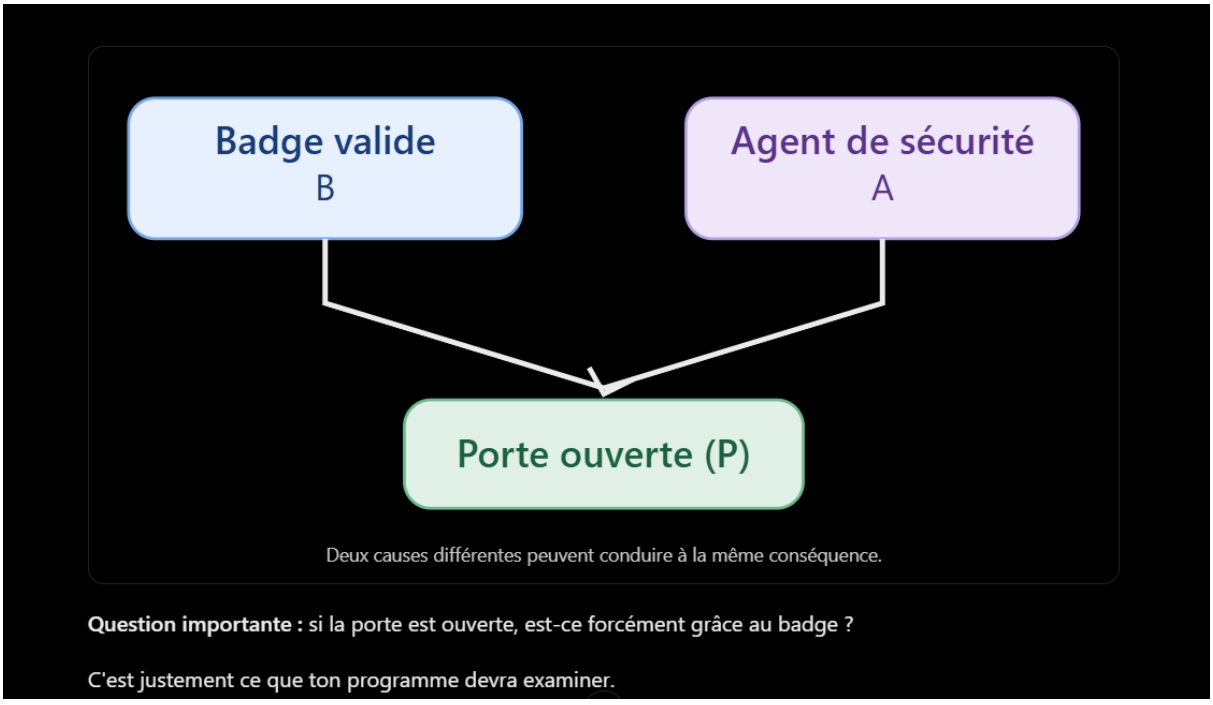
Nous avons deux règles :

$$R_1 : B \Rightarrow P$$

$$R_2 : A \Rightarrow P$$

Et un fait :

$$B = \text{Vrai}$$



À retenir : pour démontrer qu'une conclusion est obligatoire, on cherche un contre-exemple et on vérifie qu'il est impossible. Pour montrer qu'elle n'est pas obligatoire, il suffit de trouver un seul contre-exemple.
pour voir correction de lexercice sous forme pyhton :

voir Exercice02

Exercice 3 : Une chaîne de conséquences

Dernier exercice · Base de faits et base de règles

Oui ! Cet exercice est idéal pour terminer notre première partie sur les faits, les règles et la déduction logique.

Il va également nous préparer au prochain chapitre : le chaînage avant.

1. Énoncé complet — Exercice 03 du TD

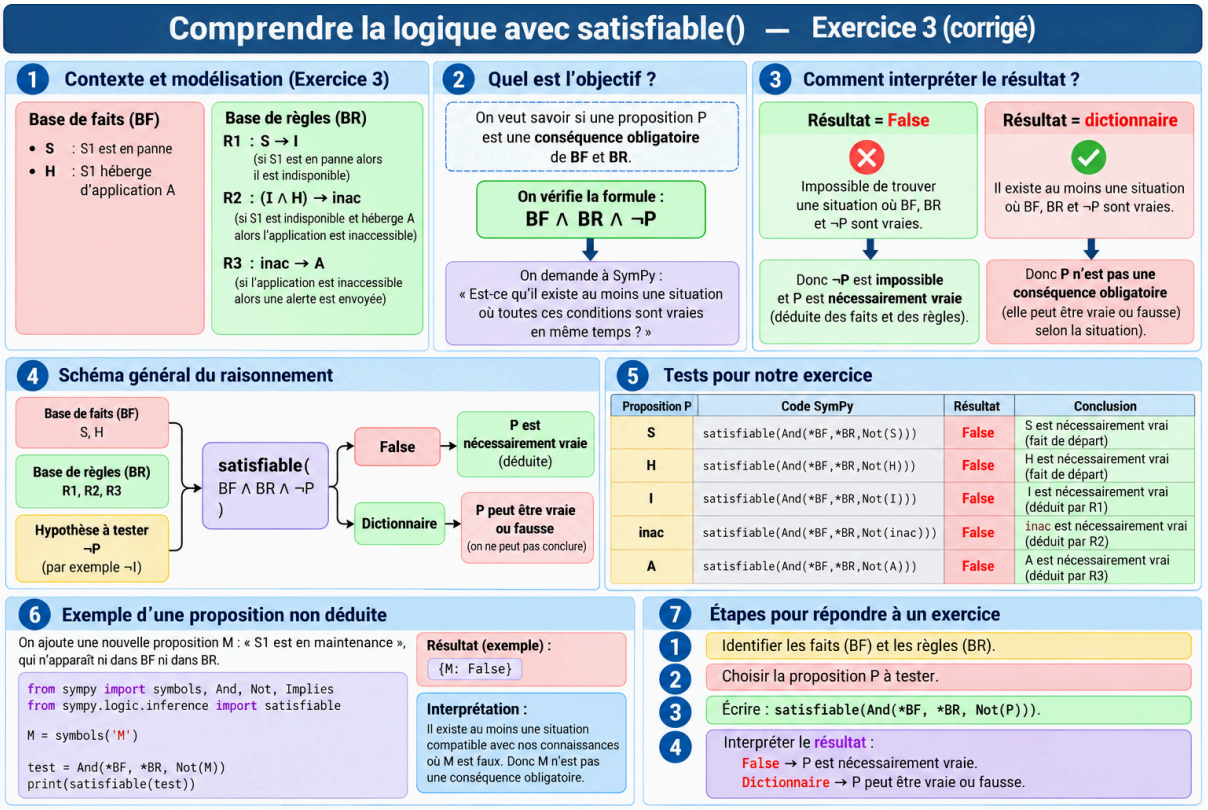
03. Une chaîne de conséquences

Faits : S1 est en panne ; S1 héberge l'application A.

Règles : R1 si un serveur est en panne, il est indisponible ; R2 si un serveur indisponible héberge une application, celle-ci est inaccessible ; R3 si une application est inaccessible, une alerte est envoyée.

Questions :

1. Écris tous les nouveaux faits que tu peux obtenir.
2. Indique à chaque fois la règle utilisée.
3. À quel moment peux-tu t'arrêter ?
   logique de raisonnement de lexercice



pour voir le code aller voir exerice3.ipynb

Complément pédagogique — Chapitre 1

Document d'accompagnement. Les cinq pages de ton cours Notion sont conservées intégralement au début du PDF. Les pages suivantes ajoutent uniquement des explications, des repères et des extraits issus de tes notebooks.

Repère de notation — exercice 1

Pour appliquer l'implication à S1, utiliser P = « S1 est en panne » et Q = « S1 est indisponible ». Sur la page 2 du document initial, Q désigne par erreur S2 : ce complément clarifie la notation sans modifier le document d'origine.

Méthode centrale : démontrer une conséquence avec SymPy

On connaît une base de faits BF et une base de règles BR. Pour vérifier si une proposition C est obligatoire, on cherche une situation compatible avec BF et BR dans laquelle C serait fausse.

```
test = And(*BF, *BR, Not(C))
resultat = satisfiable(test)
```

Résultat	Interprétation
False	Aucun contre-exemple : C découle nécessairement de BF et BR (si les connaissances sont cohérentes).
Dictionnaire	Un contre-exemple existe : C n'est pas une conséquence obligatoire.

Attention : satisfiable() vérifie une possibilité logique ; cette fonction n'ajoute pas automatiquement les faits déduits à la liste Python BF.

Exercice 1 — Exemple tiré de logiquesympy.ipynb

```
P, Q = symbols("P Q")
regle = Implies(P, Q)
fait = P
test = And(fait, regle, Not(Q))
print(satisfiable(test))  # False
```

Le test demande : « peut-on avoir S1 en panne, la règle respectée et S1 disponible ? » Non. La conclusion Q est donc nécessaire.

Exercice 2 — Badge et porte

Les deux règles sont B ⇒ P et A ⇒ P. Le seul fait initial explicite est B : Lina présente un badge valide. L'intervention de l'agent est une autre possibilité, pas un fait affirmé.

```
regle1 = Implies(B, P)
regle2 = Implies(A, P)
test1 = And(B, regle1, Not(P))
print(satisfiable(test1))  # False
```

Pour la question 2, on cherche une situation où la porte est ouverte mais le badge n'est pas valide :

```
regles = [regle1, regle2]
test3 = And(P, *regles, Not(B))
print(satisfiable(test3))  # un modèle possible
```

Un modèle possible est P vrai, B faux, A vrai : l'agent ouvre la porte sans badge valide. C'est le contre-exemple demandé.

Exercice 3 — Chaîne de conséquences

Faits initiaux : S (S1 en panne) et H (S1 héberge l'application A). Règles : S ⇒ I ; (I et H) ⇒ inac ; inac ⇒ A (alerte).

Étape	Connaissance obtenue	Règle
Initiale	S et H	Faits donnés
1	I : S1 indisponible	R1
2	inac : application inaccessible	R2, avec H
3	A : alerte envoyée	R3

```
faits = [S, H]
regles = [Implies(S,I), Implies(And(I,H),inac),
          Implies(inac,A)]
print(satisfiable(And(*faits, *regles, Not(A))))
```

Le test renvoie False : l'alerte est une conséquence obligatoire. Pour une proposition indépendante M (« S1 est en maintenance »), le test avec Not(M) peut retourner un dictionnaire : aucune règle ne force M.

Quand s'arrêter ?

Dans un raisonnement visant à produire tous les faits, on s'arrête quand aucune règle applicable n'apporte un fait nouveau : c'est le point fixe de la base de faits. Cela prépare le chapitre sur le chaînage avant et la fermeture.

Fiche express

Concept	À retenir
Fait	Information connue au départ.
Règle	Implication reliant des conditions à une conséquence.
Modus Ponens	P et (P ⇒ Q) permettent de conclure Q.
Contre-exemple	Situation compatible avec les connaissances où la conclusion est fausse.
Point fixe	Aucun fait nouveau n'est produit par les règles.

Sources et fichiers de travail

Sources : ton PDF « logique pour ia » et tes notebooks logiquesympy.ipynb, Exercice02.ipynb et exercice3.ipynb. Référence technique : documentation officielle SymPy, https://docs.sympy.org/latest/modules/logic.html .